

SAP Labs Interview Prep - Start Here & Behavioural

Nirmalya Mandal | Interview: 22 July 2026

Prepared from your resume, your projects, and the seniors' advice

Study pack - part 0 of 10

Contents

How to use this pack	3
The seniors' advice, distilled	4
Your one-line positioning	5
Self-introduction (Technical round)	6
Self-introduction (Managerial / HR round)	7
The behavioural / managerial answers	8
Why SAP?	8
Why STAR? / Why are you interested in the STAR program?	8
Where do you see yourself in 5 years?	8
How will you manage work and college / the M.Tech together?	8
Strengths and weaknesses	9
Tell me something that is not on your resume	9
Internship / experience learnings	9
Questions to ask them	11
The 60-second pre-interview checklist	12

How to use this pack

You have eleven documents. Read them in this order, but come back to whichever one you feel weak on.

0. **Start Here & Behavioural** (this file) - mindset, self-introduction, and the HR/behavioural answers.
1. **SAP & STAR Program** - what the company is, and everything about the STAR program.
2. **Projects & Experience Deep Dive** - every project explained end to end, with the tech choices.
3. **Interview Question Bank** - likely questions grouped by round, with model answers.
4. **DSA & CS Fundamentals** - the pen-and-paper C++ code and core theory.
5. **DSA - 50 Questions by Pattern** - pattern-based practice, brute force and optimised, with complexity.
6. **CS Fundamentals - DBMS, OS, CN, CD** - the core-subject question bank.
7. **SQL Practice Questions** - 30+ questions then answers, from basics to the classic interview queries.
8. **Generative AI - Question Bank** - LLMs, RAG, prompting, and the SAP Joule bridge.
9. **Managerial & HR Rounds** - the two non-technical rounds, questions with model answers.
10. **Your Self-Introduction** - human, natural intro scripts made for you.

One rule above all the others: everything written here is a *script to understand and then say in your own words*. Do not memorise these paragraphs and recite them. Your trainer said it plainly - the interviewer must not feel that you learned it from ChatGPT. So read a section, close the PDF, and say the idea out loud in your own English. If you can explain it to a friend, you own it. If you can only repeat it, you do not.

The seniors' advice, distilled

Your soft-skills trainer and the placed seniors gave a lot of advice in the group. Here is all of it compressed into things you can actually act on.

The big ones:

- **Know your resume completely.** Most questions come from your own skills, projects, and experience. If they catch you weak or lying on something you wrote, it backfires badly. Every single word on your resume is fair game - so prepare it, do not decorate it.
- **Basics matter more than depth.** They are hiring for a 2-year learning program. They want to see that your fundamentals are solid and that you can learn. You do not need to be an expert. Getting close to the right answer on the basics gets you through.
- **Show a learning attitude.** As a Scholar you will learn most of your next two years on the job. They are explicitly looking for someone eager to learn and grow, not someone who claims to already know everything.
- **Lead the interview, especially through your projects.** Do not give a bare list. Steer the conversation toward what you know deeply, and let them ask follow-ups. But never say something you cannot defend if they cross-question it.
- **Be conversational, not robotic.** Do not give one-line answers to everything, and do not deliver a memorised speech. Make it a conversation. Short, clear, simple English, but back-and-forth.
- **When you code, explain while you write.** If your logic is correct, they will overlook small syntax mistakes. Talk through what you are doing as you write it.
- **If you do not know something, say so honestly** - and add that you will look it up afterwards. One senior did exactly that and still got selected. Do not bluff.

The small but important ones:

- Pronounce it **“S-A-P”**, three separate letters. Never say the word “sap.” This is a real filter - saying “sap” signals you did no research.
- Have a general idea of **SAP S/4HANA Cloud** and **generative-AI workflows** (covered in part 1).
- For the internship / experience, do not just say what you did. Say **what you learned and what skills you gained**.
- At the end they usually ask **“Do you have any questions for us?”** - always have some ready (list at the end of this file).

Your one-line positioning

Before any answer, know the story you are selling. Everything you say should reinforce this:

A final-year CS student who does not just study software - I build and run real production applications end to end, including a healthcare app I own from the mobile app to the server it runs on, and I am hungry to learn how large-scale enterprise systems are built.

That is your theme. Builder, shipper, fast learner. Every project and every answer should quietly point back to it.

Self-introduction (Technical round)

The technical round intro should be about **your work and your technical background**. Keep it to 45-60 seconds. Structure: who you are -> what you build -> one or two proof points -> a hook they can dig into.

Say something like this (in your own words):

“Good morning. I’m Nirmalya Mandal, a final-year Computer Science student at Amity University Kolkata. I’m a full-stack developer, which means I build both the front end and the back end of applications, and I also handle deploying and running them on servers.

Most of my learning has come from building real products rather than just coursework. The one I’m proudest of is DokLink, an emergency healthcare app for finding and booking hospital beds in real time - I built the mobile app, the backend, and the infrastructure, and we’re now in the final stages before its public launch. Alongside that I’ve built e-commerce platforms, an AI system that answers questions about ocean data in plain English, and a few other production apps.

On the technical side I’m most comfortable with JavaScript and TypeScript for development, Python for backend and AI work, and C++ for data structures. I’ve worked a lot with React, Next.js, React Native, Django, and PostgreSQL, and I’ve deployed on Linux servers with Docker and Nginx.

What excites me about this role is going from building small products alone to learning how enterprise-scale systems are engineered by real teams.”

Why it works: it leads with building, names a concrete shipped product (a hook they will grab), lists your real stack without over-claiming, and ends on the learning attitude they want.

Self-introduction (Managerial / HR round)

Here the focus shifts from *what you build* to *how you work and who you are*. Keep the same length. Structure: who you are -> how you work -> what drives you -> why you fit.

Say something like this:

“I’m Nirmalya, a final-year CS student, and the short version of me is that I like taking full ownership of things. In my main role at DokLink I’m effectively the whole tech team - I own the product from design to the live server - and I’ve learned to be organised, to prioritise ruthlessly, and to keep something reliable while I’m the only one maintaining it.

I work best when I understand the *why* behind a task, not just the *what*. I’m comfortable working independently, but I’ve also worked with designers and founders on client projects, so I can take feedback and communicate clearly.

Outside the code, I’ve co-authored two research papers on human-vehicle interaction, which taught me patience and how to work in a structured team over months.

What draws me to SAP and the STAR program is that it’s built around exactly how I like to grow - real work, real responsibility, and structured learning at the same time. That’s the environment where I do my best.”

Why it works: ownership, teamwork, communication, and a clean bridge into “Why STAR.” It answers “how do you work” before they even ask.

The behavioural / managerial answers

For each: the question, what they are really testing, and a model answer to make your own. **Do not memorise - internalise.**

Why SAP?

What they test: Did you research the company, or are you here for any job? Genuine interest scores.

“A few reasons. First, the scale - SAP runs the back office of a huge share of the world’s businesses. When I read that around 98% of the S&P 500 companies are SAP customers and that a large part of global commerce touches SAP systems, it hit me that this is software operating at a scale I’ve never worked at. I’ve built apps for hundreds of users; here I’d be learning systems that serve the world’s largest companies.

Second, I’ve always worked on the full stack and on solving real business problems - my glass-manufacturing and agricultural client projects were literally about digitising how a business runs. That’s exactly what SAP does, just at enterprise scale, so it feels like a natural step up.

And third, the STAR program specifically - the idea of doing real engineering work while earning an M.Tech from BITS Pilani is the perfect way for someone like me, who learns best by building, to grow.”

Why STAR? / Why are you interested in the STAR program?

This is the single most important behavioural question. They will ask it, possibly more than once. Full treatment with a passionate model answer is in **part 1** - study it there. Short version: real work + structured learning + a BITS Pilani M.Tech + three team rotations, which is exactly how you learn best.

Where do you see yourself in 5 years?

What they test: Are you ambitious but realistic? Will you stay and grow, or leave in a year?

“In five years I’d want to have grown from someone who builds products alone into a strong engineer inside a real engineering organisation. Realistically, the first two years I’d be a Scholar - learning SAP’s technologies deeply, rotating through different teams, and finishing my M.Tech. After that, I’d want to be a solid full-time engineer who’s trusted with real ownership of a part of a product, the way I own things now but at a much bigger scale. Longer term I’d like to grow toward leading a small team or a technical area, because I already enjoy owning the whole picture of a product. But honestly, my first focus is just to learn as much as I can and become genuinely good at enterprise engineering.”

Note: mention the M.Tech and rotations - it shows you understand the program is a 2-year journey.

How will you manage work and college / the M.Tech together?

What they test: Can you handle the dual-study-and-work structure? It is the core of the program.

“I’m actually already living a version of this. Right now I’m a final-year student while also running the tech for DokLink and doing client projects on the side, so I’ve had to get good at time-boxing my work, prioritising what actually matters, and not letting either side slip. The STAR program is more

structured than my current juggling, which honestly makes it easier - it's designed so that the work and the M.Tech support each other. My approach is the same one that's worked so far: plan the week, protect focused time for deep work, and be honest early if something needs more time rather than letting it pile up."

Strengths and weaknesses

What they test: Self-awareness. A fake weakness ("I work too hard") is an instant red flag.

Strengths - pick two, back each with proof:

"My biggest strength is ownership - I don't wait to be told every step. At DokLink I'm the only developer, so I've had to take a product from an idea all the way to a finished app that's about to launch, including the parts most people avoid like servers and deployment. My second strength is that I learn fast by building - most of my stack I taught myself because a project needed it."

Weakness - real, with what you are doing about it:

"A genuine weakness is that because I mostly work alone, I sometimes go deep into building before asking whether I'm solving it the simplest way - I can over-engineer. I've become aware of it, and I've started forcing myself to step back and ask 'what's the simplest thing that works?' before writing code. Working in a team like SAP's would actually help me here, because there'd be people to sanity-check my approach early."

This is strong because the weakness is real, believable, and you frame the job as part of the fix.

Tell me something that is not on your resume

What they test: Are you a person, not a document? Can you be natural and human?

Have ONE genuine thing ready. Options that fit you:

- "One thing my resume doesn't show is that I basically learn by breaking things. A lot of what I know about servers and deployment I learned because I broke a live site once and had to fix it fast - that pressure taught me more than any tutorial."
- Or something personal and true - a hobby, how you got into coding, a habit. Keep it honest and light. The point is to sound like a person, not to impress.

Avoid turning it into another technical brag. A little bit of personality is the whole point.

Internship / experience learnings

What they test: Do you reflect and grow, or just clock hours? The seniors stressed this repeatedly - **don't say what you did, say what you learned.**

"The biggest thing my work at DokLink taught me wasn't a specific technology - it was how much harder *production* is than a class project. Anyone can make something work on their laptop. Making it work reliably for real users, handling the case where two people book the last bed at the same instant, keeping it running on a server, recovering when something breaks - that's the real engineering, and I

only learned it because I had to own it. I also learned to make trade-offs consciously - choosing the simple, reliable option over the fancy one because I'm the one who has to maintain it at 3 a.m."

This single answer signals maturity, ownership, and the learning attitude all at once.

Questions to ask them

When they ask “Do you have any questions for us?”, **always say yes**. It is a scoring opportunity, not a formality. Ask two or three, genuinely curious ones. Good options:

- “Across the three rotations in the STAR program, how are Scholars matched to teams - is it based on our interests, or on where the need is?”
- “For someone coming in strong on web and mobile but new to enterprise systems like ABAP and HANA, what does the first few months of learning usually look like?”
- “What separates a Scholar who does really well in the program from one who just gets by?”
- “What does the team I’d join actually work on day to day?”

You can also do what a senior suggested: turn it into a moment to share a learning. “Can I also mention - one thing I took away from building my own product is X” - this leaves a strong final impression.

Avoid: questions about salary, leaves, or “will I definitely get converted to full-time” as your first question. Those signal the wrong priorities. Lead with curiosity about the work and the learning.

The 60-second pre-interview checklist

Right before you walk in, remind yourself:

1. Say **S-A-P**, never “sap.”
2. Lead with **building and shipping**. Your hook is DokLink, an emergency healthcare app you built and run end to end.
3. **Basics + learning attitude** beat trying to sound like an expert.
4. **Never claim what you can’t defend**. If unsure, say so honestly.
5. When coding, **talk through the logic** as you write.
6. Keep it a **conversation**, not a speech. Breathe. Smile.
7. Have your **questions for them** ready.